Software Design Descriptions – Sistem de programare a bazei sportive universitare

1. Introducere

1.1 Scopul documentului

Acest document descrie modul în care va fi construit **CampusSport**: arhitectura, componentele, datele, interfețele și algoritmii care implementează cerințele din specificația SRS.

1.2 Domeniul

Designul acoperă aplicația web progresivă pentru utilizatori, panoul de administrare, serverul de aplicație, baza de date și integrările cu SSO, orarul facultății și serviciile de notificare. Plățile online, controlul fizic al ușilor și gestiunea echipamentelor rămân în afara acestei versiuni, ca în SRS.

1.3 Definiții suplimentare

Termenii din SRS (resursă, slot, membru, extern, blocare, no-show, EF) se păstrează. Termeni noi în acest document:

Termen	Definiție
API REST	Interfața HTTP/JSON prin care clientul comunică cu serverul
JWT	JSON Web Token – jeton semnat care identifică sesiunea utilizatorului
Worker	Proces separat care execută sarcini în fundal (notificări, remindere, no-show)
Constrângere de excludere	Regulă PostgreSQL care interzice două rânduri cu intervale suprapuse pe aceeași resursă
SSE	Server-Sent Events – canal prin care serverul trimite actualizări în timp real către browser
ORM	Strat de mapare între obiectele din cod și tabelele bazei de date

2. Considerente de proiectare

Designul pune pe primul loc corectitudinea rezervărilor (zero suprapuneri) și simplitatea pentru student; restul deciziilor decurg din aceste două priorități.

2.1 Obiective de proiectare

Obiectiv	Cum îl atinge designul	Cerințe SRS
Zero rezervări suprapuse	Constrângere de excludere în baza de date, nu doar verificare în cod	FR-12, FR-13, NFR-04
Priorități aplicate automat	Blocările (EF, evenimente, urgențe) sunt stocate în același tabel de ocupare ca rezervările	FR-19 – FR-24
Notificări rapide	Coadă de mesaje + worker dedicat, trimitere paralelă	FR-31, FR-41, NFR-03
Simplitate pe telefon	PWA cu 4 ecrane principale, fără instalare din magazin	UI-01 – UI-09
Reguli modificabile fără cod	Tabel de setări citit la fiecare cerere	FR-46, NFR-14

2.2 Constrângeri

- Găzduire în UE și minimizarea datelor personale (GDPR);
- autentificarea membrilor exclusiv prin SSO-ul universității;
- echipa de administrare are 1–2 persoane, deci sistemul trebuie să ruleze fără intervenție manuală zilnică;
- buget limitat: componente open-source, fără licențe comerciale obligatorii.

2.3 Decizii tehnologice

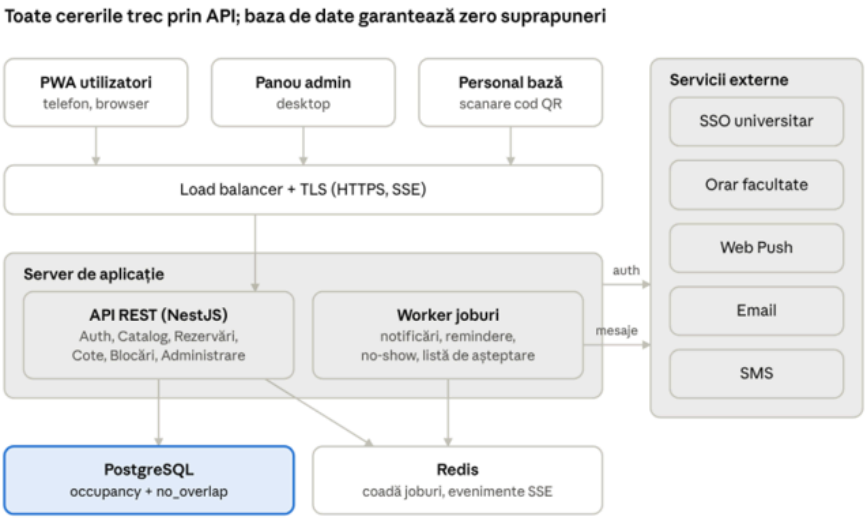
Strat	Tehnologie aleasă	Motivul alegerii
Client utilizatori	PWA în React + TypeScript	Rulează pe orice telefon din browser, instalabilă, funcționează parțial offline

Panou de administrare	Același proiect React, rute protejate pe rol	Un singur cod de întreținut
Server de aplicație	Node.js (NestJS) + TypeScript	Același limbaj ca clientul, bun pentru I/O și timp real
Bază de date	PostgreSQL 15+	Tranzacții, tipul `tstzrange` și constrângeri de excludere pentru suprapuneri
Coadă și cache	Redis + BullMQ	Joburi programate (remindere, no-show) și trimitere asincronă a notificărilor
Timp real	Server-Sent Events	Actualizarea grilei de disponibilitate fără reîncărcare
Autentificare	OpenID Connect (SSO) + JWT; cod OTP pentru externi	Membrii nu au parole proprii; externii se verifică prin email și telefon
Notificări	Web Push (VAPID), email tranzacțional, SMS prin API	Canale independente, cu rezervă SMS pentru urgențe
Găzduire	Containere Docker pe cloud UE sau pe serverele universității	Portabilitate, medii identice de test și producție

Alternativele respinse: aplicații native iOS/Android (cost dublu de dezvoltare, instalare obligatorie) și baze de date NoSQL (garanția anti-suprapunere ar trebui scrisă manual în cod).

3. Arhitectura sistemului

CampusSport folosește o arhitectură în trei straturi – client PWA, server de aplicație fără stare și bază de date relațională – cu un worker separat pentru tot ce se întâmplă în fundal.



Clienții vorbesc doar cu API-ul; API-ul scrie rezervările în PostgreSQL, unde constrângerea `no_overlap` respinge orice suprapunere, și pune sarcinile în Redis, de unde worker-ul trimite notificările prin serviciile externe.

3.1 Stiluri arhitecturale

- **Monolit modular**: un singur server cu module separate (cap. 4), mai simplu de întreținut de o echipă mică decât microserviciile;
- **Procesare asincronă**: notificările și joburile programate rulează în worker, ca o rezervare să nu aștepte trimiterea mesajelor;
- **Server fără stare**: sesiunile sunt în JWT, deci se pot adăuga instanțe API la vârfuri de trafic.

4. Proiectarea componentelor

Serverul este împărțit în 9 module cu responsabilități separate; fiecare expune servicii interne și, unde e cazul, rute API.

Modul	Responsabilitate	Depinde de	Cerințe SRS
Auth	Login SSO (OIDC), înregistrare externi cu OTP, emitere JWT, roluri	SSO, Notificări	FR-01, FR-02, FR-05
Catalog	Sporturi, resurse, dotări, program de funcționare, filtrare după sport	–	FR-06 – FR-10, FR-39

Disponibilitate	Calculează grila de sloturi libere/ocupate/blocate pentru o resursă și o zi	Catalog, Rezervări, Blocări	FR-09, FR-10
Rezervări	Creare, anulare, check-in, QR semnat, verificarea regulilor de prioritate și fereastră	Cote, Disponibilitate, Notificări	FR-11 – FR-18, FR-23
Cote	Contorul lunar per student, limita de rezervări active, restituiri	Setări	FR-25 – FR-29
Blocări	Orar EF (import CSV/iCal), evenimente, mentenanță, blocare de urgență; anularea rezervărilor afectate	Rezervări, Notificări	FR-19 – FR-22, FR-40 – FR-42
Listă de așteptare	Înscrieri, abonări la zi/sport, oferirea locurilor eliberate	Rezervări, Notificări	FR-30 – FR-33
Notificări	Generarea mesajelor, alegerea canalului, trimiterea prin push/email/SMS, urmărirea livrării	Coadă Redis, furnizori externi	FR-34 – FR-38

Administrare și rapoarte	Setări, jurnal audit, panou zilnic, rapoarte CSV/PDF	Toate modulele	FR-43 – FR-46

4.1 Joburi de fundal (worker)

Job	Declanșare	Acțiune
send-notification	La fiecare eveniment care afectează o rezervare	Trimite mesajul pe canalele utilizatorului, reîncearcă de 3 ori
reminder	Programat la −24 h și −1 h față de început	Trimite reminder cu buton de anulare
no-show-check	Programat la +15 min față de început	Marchează no-show dacă lipsește check-in-ul și eliberează slotul
waitlist-offer	La eliberarea unui slot	Oferă locul conform regulii (simultan sau pe rând, 10 min)
waitlist-expire	La 10 min după o ofertă	Trece la următorul din listă
monthly-reset	Prima zi a lunii, 00:00	Pornește contoarele noi de cote
anonymize	Zilnic, noaptea	Anonimizează rezervările mai vechi de 12 luni

4.2 Clientul (PWA)

Aplicația client are 4 ecrane pentru utilizatori – Start (alegerea sportului), Disponibilitate (grila de sloturi), Confirmare, Rezervările mele (cu QR) – și un set de ecrane de administrare: Panou zilnic, Resurse, Orar și evenimente, Blocare, Setări, Rapoarte. Un service worker păstrează local ultima rezervare și codul QR, pentru afișare fără internet.

5. Proiectarea datelor

Rezervările și blocările împart un singur tabel de ocupare (`occupancy`), astfel încât o singură constrângere a bazei de date garantează că nicio rezervare nu se suprapune cu alta sau cu o oră EF,

un eveniment ori o avarie.

5.1 Tabele

Tabel	Câmpuri principale	Observații
users	id (uuid), full_name, email, phone, role (student/teacher/external/admin), status (active/suspended), suspended_until, sso_subject, created_at	email unic; sso_subject null pentru externi
sports	id, name_ro, name_en, icon, required_equipment (text[])	ex. baschet → {panou, minge baschet}
resources	id, name, type, location, photo_url, capacity, equipment (text[]), opening_hours (jsonb), slot_minutes, external_access (never/window/leftover), active	
resource_sports	resource_id, sport_id	legătura N:N resursă–sport
occupancy	id, resource_id, period (tstzrange), kind (booking/pe_class/event/maintenance/emergency), status (active/cancelled/completed/no_show), user_id, created_by, reason, recurrence_id, cancel_source, checked_in_at, qr_nonce	tabelul central

recurrences	id, rule (RRULE iCal), resource_id, kind, valid_from, valid_to, source_file	orar EF semestrial
monthly_quota	user_id, month (date), used, refunded	cheie primară (user_id, month)
waitlist	id, user_id, resource_id (null = orice resursă compatibilă), sport_id, day, period (tstzrange, opțional), priority, status (waiting/offered/accepted/expired), offered_at	
notifications	id, user_id, occupancy_id, type, channel (push/email/sms), payload (jsonb), sent_at, delivered_at, error	
push_subscriptions	id, user_id, endpoint, keys (jsonb), created_at	un rând per dispozitiv
settings	key, value (jsonb), updated_by, updated_at	cotă, ferestre, termene
audit_log	id, actor_id, action, entity, entity_id, before (jsonb), after (jsonb), at	doar inserare, fără modificare

5.2 Constrângeri și indexuri

Garanția anti-suprapunere (FR-12) este o constrângere de excludere pe `occupancy`:

```
CREATE EXTENSION IF NOT EXISTS btree_gist;

ALTER TABLE occupancy ADD CONSTRAINT no_overlap
  EXCLUDE USING gist (
    resource_id WITH =,
```

```
    period WITH &&
  ) WHERE (status = 'active');
```

Alte reguli la nivel de date:

- index GiST pe `(resource_id, period)` pentru calculul rapid al disponibilității;
- index pe `(user_id, status, lower(period))` pentru „Rezervările mele” și limita de 2 rezervări active;
- `CHECK (kind = 'booking') = (user_id IS NOT NULL)` – orice rezervare are un titular;
- `audit_log` și `notifications` sunt partiționate lunar, pentru rapoarte și ștergere ușoară.

5.3 Păstrarea datelor

Rezervările mai vechi de 12 luni sunt anonimizate (`user_id` setat la null, păstrând statistica). Conturile externe inactive 24 de luni sunt șterse. La cerere, utilizatorul își exportă datele în JSON sau își șterge contul (NFR-11).

6. Proiectarea interfețelor

Clientul comunică cu serverul printr-un API REST JSON versionat (`/api/v1`), documentat în OpenAPI, plus un canal SSE pentru actualizări în timp real.

6.1 API REST – rute principale

Metodă	Rută	Rol minim	Descriere
GET	/auth/sso/login	public	Redirecționare către SSO-ul universității
POST	/auth/external/register	public	Înregistrare extern, trimite OTP pe email și telefon
POST	/auth/external/verify	public	Verifică OTP, emite JWT
GET	/sports	public	Lista sporturilor cu pictograme
GET	/resources?sport={id}	public	Doar resursele cu dotările necesare sportului
GET	/availability?resource=	public	Grila de sloturi; numele titularilor

	{id}&date={zi}		doar pentru admin
POST	/bookings	utilizator	Creează rezervare; 409 dacă slotul e ocupat, 422 dacă o regulă e încălcată
DELETE	/bookings/{id}	titular	Anulează și eliberează slotul
POST	/bookings/{id}/check-in	titular / personal	Check-in prin QR sau buton
GET	/me/bookings	utilizator	Rezervări viitoare și istoric
GET	/me/quota	student	Intrări folosite și rămase în luna curentă
POST	/waitlist	utilizator	Înscriere pe slot sau abonare la zi + sport
POST	/waitlist/{id}/accept	utilizator	Acceptă oferta pentru locul eliberat
POST	/me/push-subscriptions	utilizator	Înregistrează dispozitivul pentru notificări push
POST	/admin/blocks	admin	Blocare de urgență sau planificată
POST	/admin/schedule/import	admin	Import orar EF (CSV/iCal)
POST	/admin/events	admin	Rezervare pentru eveniment al

			universității
POST	/admin/announcements	admin	Anunț către utilizatorii unei resurse sau interval
GET / PUT	/admin/settings	admin	Citire și modificare reguli
GET	/admin/reports?month=	admin	Rapoarte, export CSV/PDF

Codurile de eroare au mesaje în română și engleză, scrise pentru utilizator (ex. „Ți-ai folosit cele 4 intrări din octombrie”).

6.2 Timp real

Endpoint-ul `GET /stream/availability?resource={id}&date={zi}` (SSE) trimite evenimentul `slot-changed` la fiecare creare, anulare sau blocare, astfel încât grila de pe telefon se actualizează în sub 3 secunde (SRS 8.3.4).

6.3 Interfețe externe

Sistem	Mod de integrare
SSO universitar	OpenID Connect, flux Authorization Code + PKCE; rolul citit dintr-un claim (ex. `eduPersonAffiliation`)
Orar facultate	Import fișier CSV/iCal sau citire periodică de la un URL iCal
Web Push	Protocol Web Push cu chei VAPID
Email	API tranzacțional sau SMTP, șabloane HTML RO/EN
SMS	API REST al furnizorului, folosit doar pentru urgențe și utilizatori fără push
Calendar personal	Fișier .ics per rezervare și abonament iCal personal

6.4 Interfața cu utilizatorul

Ecranele urmează SRS 8.3.1: pictograme mari pe sporturi, grilă orizontală de sloturi glisabilă, buton de confirmare în partea de jos a ecranului (zona degetului mare). Sistemul de design folosește o paletă sportivă cu contrast AA, componente de minimum 44 px și un singur font sans-serif.

7. Algoritmi și fluxuri cheie

7.1 Crearea unei rezervări (fără conflicte)

Toate verificările și scrierea se fac într-o singură tranzacție; dacă două cereri sosesc simultan, constrângerea `no_overlap` lasă să treacă exact una.

1. Verifică JWT-ul și starea contului (nesuspendat).
2. Verifică fereastra de rezervare după rol: student 7 zile, profesor 14 zile, extern 24 h.
3. Pentru externi: verifică `external_access` al resursei; pentru resursele „leftover”, slotul trebuie să înceapă în următoarele 24 h.
4. Pentru studenți: blochează rândul din `monthly_quota` (`SELECT … FOR UPDATE`); dacă `used - refunded ≥ 4` și slotul e la peste 24 h → eroare 422 „cotă epuizată”.
5. Verifică limita de 2 rezervări active viitoare.
6. Inserează în `occupancy` cu `status = 'active'` și un `qr_nonce` aleator.
7. Dacă baza de date respinge cu încălcarea `no_overlap` → rollback și răspuns 409 „Slot ocupat”, cu oferta de listă de așteptare.
8. Incrementează `used` , scrie în `audit_log` , face commit.
9. După commit: publică `slot-changed` (SSE), pune în coadă confirmarea și programează joburile `reminder` și `no-show-check` .

7.2 Eliberarea unui slot și lista de așteptare

La orice anulare, no-show sau deblocare, modulul Listă de așteptare caută înscrierile potrivite (aceeași resursă și interval sau abonare la zi + sport cu resursă compatibilă) și le sortează după prioritate: profesor, student, extern, apoi după ora înscrierii.

- Dacă slotul începe în sub 2 ore: toți primesc notificarea simultan; primul `POST /waitlist/{id}/accept` creează rezervarea prin algoritmul 7.1, ceilalți primesc „Locul a fost luat”.
- Altfel: oferta merge la primul din listă (`status = 'offered'`); jobul `waitlist-expire` o trece la următorul după 10 minute fără răspuns.

Acceptarea trece tot prin 7.1, deci constrângerea de excludere rezolvă și aici cererile simultane.

7.3 Blocare de urgență

1. Administratorul trimite `POST /admin/blocks` cu resursa, intervalul (sau fără sfârșit) și motivul.
2. Într-o tranzacție: selectează rezervările active care se suprapun, le setează `status = 'cancelled'` , `cancel_source = 'emergency'` , restituie cota (`refunded + 1`) și

inserează blocarea.

3. După commit: pentru fiecare utilizator afectat se pune în coadă o notificare cu prioritate maximă pe toate canalele (push + email + SMS dacă nu are push), cu 2–3 alternative calculate (aceeași oră, altă resursă compatibilă).
4. Worker-ul trimite în paralel (minimum 50 de mesaje pe secundă); panoul de administrare afișează contorul „anunțați / livrați”.

Obiectivul este livrarea către toți cei afectați în sub 2 minute (NFR-03).

7.4 Import orar EF și evenimente

Fișierul CSV/iCal este validat, transformat în reguli de recurență și extins în intervale pe semestru. Sistemul calculează mai întâi conflictele cu rezervările existente și le arată administratorului; după confirmare, aplică același pas 2–3 din 7.3, cu `cancel_source = 'pe_class'` sau `'event'`. Evenimentele pot fi programate oricât de departe în viitor, fără limita de fereastră a utilizatorilor.

7.5 Check-in și no-show

Codul QR conține `occupancy_id` + `qr_nonce`, semnate HMAC cu cheia serverului; personalul îl scanează, iar serverul verifică semnătura, ziua și fereastra −10 / +15 minute. Jobul `no-show-check` rulează la +15 minute: fără check-in, rezervarea devine `no_show`, intrarea rămâne consumată și se pornește 7.2. La al 3-lea no-show într-o lună contul este suspendat 14 zile.

8. Securitate, implementare și trasabilitate

8.1 Securitate

Aspect	Soluție de proiectare
Autentificare	OIDC pentru membri; OTP pe email și telefon pentru externi; 2FA obligatoriu pentru administratori
Sesiuni	JWT de acces cu durată 15 min + token de reîmprospătare (30 zile pe telefon, 8 h pentru admin), stocat în cookie HttpOnly, Secure, SameSite=Strict
Autorizare	Control pe roluri (RBAC) verificat pe server la fiecare rută; un utilizator vede doar rezervările proprii
Coduri QR	Semnătură HMAC-SHA256, valabile doar ziua rezervării, invalidate la anulare

Protecție API	Limitare de cereri (ex. 10 rezervări/minut per utilizator), validarea tuturor intrărilor, interogări parametrizate prin ORM
Date	TLS 1.2+ în tranzit; criptare a discului bazei de date; secrete în manager de secrete, nu în cod
Audit	Toate acțiunile administrative scrise în `audit_log`, doar inserare
Testare	Scanare OWASP Top 10 și test de penetrare înainte de lansare

8.2 Implementare (deployment)

Componentă	Instanțe	Note
Reverse proxy / load balancer	1 (gestionat)	Terminare TLS, servește fișierele statice ale PWA
Server API (NestJS)	2+ containere	Fără stare; scalare orizontală
Worker notificări și joburi	1–2 containere	Citește din coada Redis
PostgreSQL	1 primar + 1 replică	Backup zilnic, păstrat 30 de zile, în altă locație
Redis	1	Coadă de joburi și distribuția evenimentelor SSE între instanțe
Monitorizare	–	Metrici, loguri centralizate, alertă dacă notificările întârzie peste 2 minute

Sunt trei medii separate – dezvoltare, testare, producție – cu livrare automată prin CI/CD după trecerea testelor (inclusiv testul de concurență cu 100 de cereri simultane pe același slot).

8.3 Trasabilitate SRS → design

Cerințe SRS	Elemente de design
FR-01 – FR-05 (acces și identificare)	Modul Auth, 6.3 SSO, 7.5 QR, 8.1
FR-06 – FR-10 (căutare după sport)	Modul Catalog, tabele `sports` / `resource_sports`, rută `/resources?sport=`
FR-11 – FR-18 (rezervări fără conflicte)	Algoritmul 7.1, constrângerea `no_overlap`, 6.2 SSE
FR-19 – FR-24 (priorități EF, evenimente)	Modul Blocări, tabel `recurrences`, 7.4
FR-25 – FR-29 (limita lunară)	Modul Cote, tabel `monthly_quota`, pasul 4 din 7.1
FR-30 – FR-33 (listă de așteptare)	Modul Listă de așteptare, 7.2, joburile `waitlist-offer` / `waitlist-expire`
FR-34 – FR-38 (notificări)	Modul Notificări, worker, tabel `notifications`, 6.3
FR-39 – FR-46 (administrare)	Modul Administrare, 7.3, `settings`, `audit_log`
UI-01 – UI-09 (interfață)	4.2 Clientul PWA, 6.4
NFR-01 – NFR-15	2.3 Decizii tehnologice, 5.2 indexuri, 8.1 Securitate, 8.2 Implementare
SYS-01 – SYS-10	8.2 Implementare